

Lab 12 - Relatório

Introdução

O objetivo do laboratório 12 é implementar três APIs REST em python, sendo elas: Usuários, Produtos e Pedidos. A API REST “Pedidos” foi criada após as outras devido ao último exercício pedido. Foi criado, também, o teste_cliente.py para testar todas as APIs que foram criadas (com métodos como POST, GET, PUT e DELETE), também foi utilizada a ferramenta Curl para realizar inserções diretas dos métodos sem precisar realizar código. Também foram tirados prints do wireshark a partir do loopback (pois tudo está rodando localmente). O laboratório foi um sucesso apesar da complexidade que ele apresenta.

Desenvolvimento

Para encurtar o laboratório, os códigos estão em anexo na pasta .zip desse arquivo enviado no canvas. Para analisar como tudo foi feito e ocorreu, analisaremos o comportamento de cada código utilizado, para demonstrar o sucesso de cada API com prints do terminal e do wireshark, a explicação estará no cabeçalho do print.

Comportamento api_usuarios.py

A api_usuarios.py tem o objetivo de guardar os usuários que poderão ser utilizados em outras APIs e para o teste que iremos fazer.

PRINT 1 e 2: Os dois primeiros prints abaixo demonstram com clareza que a utilização dos métodos foi um sucesso nessa API Usuários (PUT, DELETE, GET e

POST), uma vez ela ficou ativa e o teste cliente fez o seu dever e a ferramenta Curl também foi utilizada.

```
allan-samuel@allan-samuel-VirtualBox:~/python$ /bin/python3 /home/allan-samuel/python/api_usuarios.py
Traceback (most recent call last):
  File "/home/allan-samuel/python/api_usuarios.py", line 2, in <module>
    from flask import Flask, jsonify, request
ModuleNotFoundError: No module named 'flask'
allan-samuel@allan-samuel-VirtualBox:~/python$ source venv/bin/activate
(venv) allan-samuel@allan-samuel-VirtualBox:~/python$ python3 api_usuarios.py
* Serving Flask app 'api_usuarios'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5001
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 235-669-078
127.0.0.1 - - [30/Oct/2025 08:54:30] "GET /usuarios HTTP/1.1" 200 -
127.0.0.1 - - [30/Oct/2025 08:54:30] "POST /usuarios HTTP/1.1" 201 -
127.0.0.1 - - [30/Oct/2025 08:54:30] "POST /usuarios HTTP/1.1" 400 -
127.0.0.1 - - [30/Oct/2025 08:54:30] "GET /usuarios HTTP/1.1" 200 -
127.0.0.1 - - [30/Oct/2025 08:55:07] "GET /usuarios/1 HTTP/1.1" 200 -
127.0.0.1 - - [30/Oct/2025 09:03:49] "PUT /usuarios/1 HTTP/1.1" 200 -
```

```
allan-samuel@allan-samuel-VirtualBox:~/python$ source venv/bin/activate
(venv) allan-samuel@allan-samuel-VirtualBox:~/python$ python3 api_usuarios.py
* Serving Flask app 'api_usuarios'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5001
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 143-102-710
127.0.0.1 - - [30/Oct/2025 09:59:10] "GET /usuarios HTTP/1.1" 200 -
127.0.0.1 - - [30/Oct/2025 09:59:10] "POST /usuarios HTTP/1.1" 201 -
127.0.0.1 - - [30/Oct/2025 09:59:10] "POST /usuarios HTTP/1.1" 400 -
127.0.0.1 - - [30/Oct/2025 09:59:10] "GET /usuarios HTTP/1.1" 200 -
127.0.0.1 - - [30/Oct/2025 09:59:46] "DELETE /usuarios/1 HTTP/1.1" 204 -
```

PRINT 3: o print abaixo mostra a utilização da ferramenta Curl, apesar do teste cliente também fazer uma parte dos métodos para teste.

```
allan-samuel@allan-samuel-VirtualBox:~$ curl -X POST http://localhost:5001/usuarios -H "Content-Type: application/json" -d '{"nome":"João Lima","email":"joao@exemplo.com"}'
{"email": "joao@exemplo.com",
 "id": 4,
 "nome": "Jo\u00e3o Lima"
}
```

PRINT 4: O print abaixo demonstra um GET para a api_usuarios utilizando a ferramenta Curl:

```
To stop seeing this message, run the following command:
curl.snap-acked

[
  {
    "email": "alice.s@exemplo.com",
    "id": 1,
    "nome": "Alice Silva"
  },
  {
    "email": "bruno.c@exemplo.com",
    "id": 2,
    "nome": "Bruno Costa"
  },
  {
    "email": "carlos.v@exemplo.com",
    "id": 3,
    "nome": "Carlos Vianna"
  }
]
allan-samuel@allan-samuel-VirtualBox:~$
```

PRINT 5: O print abaixo demonstra um PUT, ou seja, uma atualização do usuário Alice.

```
allan-samuel@allan-samuel-VirtualBox:~$
curl -X PUT http://localhost:5001/usuarios/1 -H "Content-Type: application/json" -d '{"nome":"Alice Modificada"}'
{
  "email": "alice.s@exemplo.com",
  "id": 1,
  "nome": "Alice Modificada"
}
allan-samuel@allan-samuel-VirtualBox:~$
```

PRINT 6: O print abaixo demonstra a utilização da ferramenta Curl para um DELETE na api_usuarios e api_produtos.

```
allan-samuel@allan-samuel-VirtualBox: ~
allan-samuel@allan-samuel-VirtualBox:~$ curl -X DELETE -i http://localhost:5001/usuarios/1
HTTP/1.1 204 NO CONTENT
Server: Werkzeug/3.1.3 Python/3.12.3
Date: Thu, 30 Oct 2025 12:59:46 GMT
Content-Type: text/html; charset=utf-8
Connection: close

allan-samuel@allan-samuel-VirtualBox:~$ curl -X DELETE -i http://localhost:5002/produtos/101
HTTP/1.1 204 NO CONTENT
Server: Werkzeug/3.1.3 Python/3.12.3
Date: Thu, 30 Oct 2025 12:59:57 GMT
Content-Type: text/html; charset=utf-8
Connection: close

allan-samuel@allan-samuel-VirtualBox:~$
```

PRINTS DO WIRESHARK: Os prints abaixo demonstram como foi um sucesso todas as tentativas de uso dos métodos na api_usuarios.

/tmp/wireshark_loP3BMF3.pcapng 260 total packets, 23 shown

```
No.      Time      Source      Destination      Protocol Length Info
40 87.061442144 127.0.0.1 127.0.0.1 HTTP 219 GET /usuarios HTTP/1.1
Frame 40: 219 bytes on wire (1752 bits), 219 bytes captured (1752 bits) on interface lo, id 0
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 58074, Dst Port: 5001, Seq: 1, Ack: 1, Len: 153
Hypertext Transfer Protocol
No.      Time      Source      Destination      Protocol Length Info
44 87.063067913 127.0.0.1 127.0.0.1 HTTP/JSON 237 HTTP/1.1 200 OK , JSON
(application/json)
Frame 44: 237 bytes on wire (1896 bits), 237 bytes captured (1896 bits) on interface lo, id 0
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 5001, Dst Port: 58074, Seq: 167, Ack: 154, Len: 171
[2 Reassembled TCP Segments (337 bytes): #42(166), #44(171)]
Hypertext Transfer Protocol
JavaScript Object Notation: application/json
No.      Time      Source      Destination      Protocol Length Info
54 87.064757170 127.0.0.1 127.0.0.1 HTTP/JSON 124 POST /usuarios HTTP/
1.1 , JSON (application/json)
Frame 54: 124 bytes on wire (992 bits), 124 bytes captured (992 bits) on interface lo, id 0
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 58090, Dst Port: 5001, Seq: 207, Ack: 1, Len: 58
[2 Reassembled TCP Segments (264 bytes): #52(206), #54(58)]
Hypertext Transfer Protocol
JavaScript Object Notation: application/json
No.      Time      Source      Destination      Protocol Length Info
58 87.066172482 127.0.0.1 127.0.0.1 HTTP/JSON 142 HTTP/1.1 201 CREATED ,
JSON (application/json)
Frame 58: 142 bytes on wire (1136 bits), 142 bytes captured (1136 bits) on interface lo, id 0
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 5001, Dst Port: 58090, Seq: 171, Ack: 265, Len: 76
[2 Reassembled TCP Segments (246 bytes): #56(170), #58(76)]
Hypertext Transfer Protocol
JavaScript Object Notation: application/json
No.      Time      Source      Destination      Protocol Length Info
68 87.067306398 127.0.0.1 127.0.0.1 HTTP/JSON 84 POST /usuarios HTTP/
1.1 , JSON (application/json)
Frame 68: 84 bytes on wire (672 bits), 84 bytes captured (672 bits) on interface lo, id 0
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 58100, Dst Port: 5001, Seq: 207, Ack: 1, Len: 18
[2 Reassembled TCP Segments (224 bytes): #66(206), #68(18)]
Hypertext Transfer Protocol
JavaScript Object Notation: application/json
No.      Time      Source      Destination      Protocol Length Info
80 87.069624291 127.0.0.1 127.0.0.1 HTTP 219 GET /usuarios HTTP/1.1
Frame 80: 219 bytes on wire (1752 bits), 219 bytes captured (1752 bits) on interface lo, id 0
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 58112, Dst Port: 5001, Seq: 1, Ack: 1, Len: 153
Hypertext Transfer Protocol
No.      Time      Source      Destination      Protocol Length Info
84 87.070799968 127.0.0.1 127.0.0.1 HTTP/JSON 324 HTTP/1.1 200 OK , JSON
(application/json)
Frame 84: 324 bytes on wire (2592 bits), 324 bytes captured (2592 bits) on interface lo, id 0
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 5001, Dst Port: 58112, Seq: 167, Ack: 154, Len: 258
[2 Reassembled TCP Segments (424 bytes): #82(166), #84(258)]
Hypertext Transfer Protocol
JavaScript Object Notation: application/json
No.      Time      Source      Destination      Protocol Length Info
155 123.873697068 127.0.0.1 127.0.0.1 HTTP 221 GET /usuarios/1 HTTP/
1.1
Frame 155: 221 bytes on wire (1768 bits), 221 bytes captured (1768 bits) on interface lo, id 0
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 48262, Dst Port: 5001, Seq: 1, Ack: 1, Len: 155
Hypertext Transfer Protocol
No.      Time      Source      Destination      Protocol Length Info
159 123.874802055 127.0.0.1 127.0.0.1 HTTP/JSON 139 HTTP/1.1 200 OK , JSON
(application/json)
Frame 159: 139 bytes on wire (1112 bits), 139 bytes captured (1112 bits) on interface lo, id 0
```

/tmp/wireshark_loP3BMF3.pcapng 260 total packets, 23 shown

Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 5001, Dst Port: 48262, Seq: 166, Ack: 156, Len: 73
[2 Reassembled TCP Segments (238 bytes): #157(165), #159(73)]
Hypertext Transfer Protocol
JavaScript Object Notation: application/json
No. Time Source Destination Protocol Length Info
244 645.261431423 127.0.0.1 127.0.0.1 HTTP/JSON 232 PUT /usuarios/1 HTTP/
1.1 , JSON (application/json)
Frame 244: 232 bytes on wire (1856 bits), 232 bytes captured (1856 bits) on interface lo, id 0
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 52234, Dst Port: 5001, Seq: 1, Ack: 1, Len: 166
Hypertext Transfer Protocol
JavaScript Object Notation: application/json
No. Time Source Destination Protocol Length Info
248 645.263155243 127.0.0.1 127.0.0.1 HTTP/JSON 144 HTTP/1.1 200 OK , JSON
(application/json)
Frame 248: 144 bytes on wire (1152 bits), 144 bytes captured (1152 bits) on interface lo, id 0
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 5001, Dst Port: 52234, Seq: 166, Ack: 167, Len: 78
[2 Reassembled TCP Segments (243 bytes): #246(165), #248(78)]
Hypertext Transfer Protocol
JavaScript Object Notation: application/json

/tmp/wireshark_loORMBF3.pcapng 170 total packets, 20 shown

No.	Time	Source	Destination	Protocol	Length	Info
150	146.422884066	127.0.0.1	127.0.0.1	HTTP	156	DELETE /usuarios/1
HTTP/1.1						
Frame 150: 156 bytes on wire (1248 bits), 156 bytes captured (1248 bits) on interface lo, id 0						
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)						
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1						
Transmission Control Protocol, Src Port: 42628, Dst Port: 5001, Seq: 1, Ack: 1, Len: 90						
Hypertext Transfer Protocol						
No.	Time	Source	Destination	Protocol	Length	Info
152	146.424520182	127.0.0.1	127.0.0.1	HTTP	227	HTTP/1.1 204 NO CONTENT
Frame 152: 227 bytes on wire (1816 bits), 227 bytes captured (1816 bits) on interface lo, id 0						
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)						
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1						
Transmission Control Protocol, Src Port: 5001, Dst Port: 42628, Seq: 1, Ack: 91, Len: 161						
Hypertext Transfer Protocol						
No.	Time	Source	Destination	Protocol	Length	Info
162	157.133345717	127.0.0.1	127.0.0.1	HTTP	158	DELETE /produtos/101
HTTP/1.1						
Frame 162: 158 bytes on wire (1264 bits), 158 bytes captured (1264 bits) on interface lo, id 0						
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)						
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1						
Transmission Control Protocol, Src Port: 54936, Dst Port: 5002, Seq: 1, Ack: 1, Len: 92						
Hypertext Transfer Protocol						
No.	Time	Source	Destination	Protocol	Length	Info
164	157.134906861	127.0.0.1	127.0.0.1	HTTP	227	HTTP/1.1 204 NO CONTENT
Frame 164: 227 bytes on wire (1816 bits), 227 bytes captured (1816 bits) on interface lo, id 0						
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)						
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1						
Transmission Control Protocol, Src Port: 5002, Dst Port: 54936, Seq: 1, Ack: 93, Len: 161						
Hypertext Transfer Protocol						

Comportamento api_produtos.py

A `api_produtos.py` tem o objetivo de guardar os produtos que poderão ser utilizados pelos usuários entre outras ações, ou seja, talvez seja a api que mais será utilizada entre as APIs.

PRINT 1: O print abaixo demonstra com clareza que a utilização dos métodos foi um sucesso nessa API Produtos (GET, POST), uma vez ela ficou ativa e o teste cliente fez o seu dever e a ferramenta Curl também foi utilizada.

```
allan-samuel@allan-samuel-VirtualBox:~/python$ source venv/bin/activate
(venv) allan-samuel@allan-samuel-VirtualBox:~/python$ python3 api_produtos.py
* Serving Flask app 'api_produtos'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5002
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 235-669-078
127.0.0.1 - - [30/Oct/2025 08:54:30] "GET /produtos HTTP/1.1" 200 -
127.0.0.1 - - [30/Oct/2025 08:54:30] "POST /produtos HTTP/1.1" 201 -
127.0.0.1 - - [30/Oct/2025 08:54:30] "POST /produtos HTTP/1.1" 400 -
127.0.0.1 - - [30/Oct/2025 08:54:30] "GET /produtos HTTP/1.1" 200 -
127.0.0.1 - - [30/Oct/2025 08:55:07] "GET /produtos/101 HTTP/1.1" 200 -
[]
```

PRINT 2: Utilizando a ferramenta Curl, foi feito um GET para analisar quais eram os produtos cadastrados no teste_cliente.

```
root@allan-samuel-VirtualBox:/home/allan-samuel# curl -X POST http://localhost:5002/produtos '{"nome":"João Lima","email":"joao@exemplo.com"}'
curl: option -d: requires parameter
curl: try 'curl --help' or 'curl --manual' for more information
{"nome":"João Lima","email":"joao@exemplo.com"}: comando não encontrado
root@allan-samuel-VirtualBox:/home/allan-samuel# curl http://localhost:5002/produtos
[
  {
    "id": 101,
    "nome": "Notebook",
    "preco": 3500.0
  },
  {
    "id": 102,
    "nome": "Mouse Sem Fio",
    "preco": 120.5
  },
  {
    "id": 103,
    "nome": "Monitor Ultra Wide",
    "preco": 1850.99
  }
]
```

PRINT 3: Foi feito um POST na `api_produtos` utilizando a ferramenta Curl, o resultado está abaixo:

```
allan-samuel@allan-samuel-VirtualBox: $ curl -X POST http://localhost:5002/produtos -H "Content-Type: application/json" -d '{"nome":"Monitor","preco":1200.00}'
{"id": 104,
 "nome": "Monitor",
 "preco": 1200.0
}
allan-samuel@allan-samuel-VirtualBox: $
```

PRINT 4: O print abaixo mostra que foi feito um DELETE com o Curl na api_produtos (PRINT 6 do comportamento da api_usuarios).

```
* Running on http://127.0.0.1:5002
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 143-102-710
127.0.0.1 - - [30/Oct/2025 09:59:10] "GET /produtos HTTP/1.1" 200 -
127.0.0.1 - - [30/Oct/2025 09:59:10] "POST /produtos HTTP/1.1" 201 -
127.0.0.1 - - [30/Oct/2025 09:59:10] "POST /produtos HTTP/1.1" 400 -
127.0.0.1 - - [30/Oct/2025 09:59:10] "GET /produtos HTTP/1.1" 200 -
127.0.0.1 - - [30/Oct/2025 09:59:57] "DELETE /produtos/101 HTTP/1.1" 204 -
```

PRINTS DO WIRESHARK: Os prints do wireshark abaixo demonstram o sucesso das implementações dos métodos com o teste_cliente e utilização do Curl.

/tmp/wireshark_loP3BMF3.pcapng 260 total packets, 23 shown

No.	Time	Source	Destination	Protocol	Length	Info
92	87.072110117	127.0.0.1	127.0.0.1	HTTP	219	GET /produtos HTTP/1.1
Frame 92: 219 bytes on wire (1752 bits), 219 bytes captured (1752 bits) on interface lo, id 0						
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)						
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1						
Transmission Control Protocol, Src Port: 54148, Dst Port: 5002, Seq: 1, Ack: 1, Len: 153						
Hypertext Transfer Protocol						
96	87.074118237	127.0.0.1	127.0.0.1	HTTP/JSON	209	HTTP/1.1 200 OK , JSON
(application/json)						
Frame 96: 209 bytes on wire (1672 bits), 209 bytes captured (1672 bits) on interface lo, id 0						
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)						
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1						
Transmission Control Protocol, Src Port: 5002, Dst Port: 54148, Seq: 167, Ack: 154, Len: 143						
[2 Reassembled TCP Segments (309 bytes): #94(166), #96(143)]						
Hypertext Transfer Protocol						
JavaScript Object Notation: application/json						
106	87.075568267	127.0.0.1	127.0.0.1	HTTP/JSON	114	POST /produtos HTTP/1.1 , JSON (application/json)
Frame 106: 114 bytes on wire (912 bits), 114 bytes captured (912 bits) on interface lo, id 0						
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)						
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1						
Transmission Control Protocol, Src Port: 54160, Dst Port: 5002, Seq: 207, Ack: 1, Len: 48						
[2 Reassembled TCP Segments (254 bytes): #104(206), #106(48)]						
Hypertext Transfer Protocol						
JavaScript Object Notation: application/json						
110	87.076672788	127.0.0.1	127.0.0.1	HTTP/JSON	134	HTTP/1.1 201 CREATED , JSON (application/json)
Frame 110: 134 bytes on wire (1072 bits), 134 bytes captured (1072 bits) on interface lo, id 0						
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)						
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1						
Transmission Control Protocol, Src Port: 5002, Dst Port: 54160, Seq: 171, Ack: 255, Len: 68						
[2 Reassembled TCP Segments (238 bytes): #108(170), #110(68)]						
Hypertext Transfer Protocol						
JavaScript Object Notation: application/json						
120	87.077813104	127.0.0.1	127.0.0.1	HTTP/JSON	81	POST /produtos HTTP/1.1 , JSON (application/json)
Frame 120: 81 bytes on wire (648 bits), 81 bytes captured (648 bits) on interface lo, id 0						
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)						
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1						
Transmission Control Protocol, Src Port: 54176, Dst Port: 5002, Seq: 207, Ack: 1, Len: 15						
[2 Reassembled TCP Segments (221 bytes): #118(206), #120(15)]						
Hypertext Transfer Protocol						
JavaScript Object Notation: application/json						
132	87.080105854	127.0.0.1	127.0.0.1	HTTP	219	GET /produtos HTTP/1.1
Frame 132: 219 bytes on wire (1752 bits), 219 bytes captured (1752 bits) on interface lo, id 0						
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)						
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1						
Transmission Control Protocol, Src Port: 54192, Dst Port: 5002, Seq: 1, Ack: 1, Len: 153						
Hypertext Transfer Protocol						
136	87.080973319	127.0.0.1	127.0.0.1	HTTP/JSON	288	HTTP/1.1 200 OK , JSON (application/json)
Frame 136: 288 bytes on wire (2304 bits), 288 bytes captured (2304 bits) on interface lo, id 0						
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)						
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1						
Transmission Control Protocol, Src Port: 5002, Dst Port: 54192, Seq: 167, Ack: 154, Len: 222						
[2 Reassembled TCP Segments (388 bytes): #134(166), #136(222)]						
Hypertext Transfer Protocol						
JavaScript Object Notation: application/json						
167	123.876307097	127.0.0.1	127.0.0.1	HTTP	223	GET /produtos/101 HTTP/1.1
Frame 167: 223 bytes on wire (1784 bits), 223 bytes captured (1784 bits) on interface lo, id 0						
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)						
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1						
Transmission Control Protocol, Src Port: 36424, Dst Port: 5002, Seq: 1, Ack: 1, Len: 157						
Hypertext Transfer Protocol						
171	123.877573582	127.0.0.1	127.0.0.1	HTTP/JSON	123	HTTP/1.1 200 OK , JSON (application/json)
Frame 171: 123 bytes on wire (984 bits), 123 bytes captured (984 bits) on interface lo, id 0						

/tmp/wireshark_loP3BMF3.pcapng 260 total packets, 23 shown

Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 5002, Dst Port: 36424, Seq: 166, Ack: 158, Len: 57
[2 Reassembled TCP Segments (222 bytes): #169(165), #171(57)]
Hypertext Transfer Protocol
JavaScript Object Notation: application/json
No. Time Source Destination Protocol Length Info
178 123.878791465 127.0.0.1 127.0.0.1 HTTP/JSON 280 HTTP/1.1 201 CREATED ,
JSON (application/json)
Frame 178: 280 bytes on wire (2240 bits), 280 bytes captured (2240 bits) on interface lo, id 0
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 5003, Dst Port: 38420, Seq: 172, Ack: 206, Len: 214
[2 Reassembled TCP Segments (385 bytes): #176(171), #178(214)]
Hypertext Transfer Protocol
JavaScript Object Notation: application/json

Comportamento api_pedidos.py

A api_pedidos.py tem o objetivo de guardar pedidos, que poderão ser utilizados pelos usuários, ela é a mais completa pois carrega o nome dos usuários, o produto e, principalmente, o pedido.

PRINT 1: Essa foi a api criada a partir do último exercício do laboratório, a api_pedidos. O objetivo dessa API é especificar nela o pedido que será criado, ou seja, o id do produto, a quantidade do produto, o preço do produto e para quem é o produto. Foi colocado um pedido utilizando o Curl, os prints abaixo demonstram o sucesso dessa operação.

```
allan-samuel@allan-samuel-VirtualBox:~/python$ source venv/bin/activate
(venv) allan-samuel@allan-samuel-VirtualBox:~/python$ python3 api_pedidos.py
* Serving Flask app 'api_pedidos'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5003
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 235-669-078
127.0.0.1 - - [30/Oct/2025 08:55:07] "POST /pedidos HTTP/1.1" 201 -
[]
```

PRINT 2: Curl utilizado para fazer o POST do pedido.

```
allan-samuel@allan-samuel-VirtualBox:~$ curl -X POST http://localhost:5003/pedidos -H "Content-Type: application/json" -d '{"usuario_id": 1, "itens": [{"produto_id": 101, "quantidade": 1} ]}'
{"id": 1,
 "itens": [
  {
    "nome": "Notebook",
    "produto_id": 101,
    "quantidade": 1,
    "subtotal": 3500.0
  }
 ],
 "nome_usuario": "Alice Silva",
 "total": 3500.0,
 "usuario_id": 1
}
```

PRINTS DO WIRESHARK: Os prints do wireshark abaixo demonstram o sucesso de toda essa operação.

/tmp/wireshark_loP3BMF3.pcapng 260 total packets, 23 shown

No.	Time	Source	Destination	Protocol	Length	Info
150	123.870771484	127.0.0.1	127.0.0.1	HTTP/JSON	271	POST /pedidos HTTP/1.1
, JSON (application/json)						
Frame 150: 271 bytes on wire (2168 bits), 271 bytes captured (2168 bits) on interface lo, id 0						
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)						
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1						
Transmission Control Protocol, Src Port: 38420, Dst Port: 5003, Seq: 1, Ack: 1, Len: 205						
Hypertext Transfer Protocol						
JavaScript Object Notation: application/json						
178	123.878791465	127.0.0.1	127.0.0.1	HTTP/JSON	280	HTTP/1.1 201 CREATED ,
JSON (application/json)						
Frame 178: 280 bytes on wire (2240 bits), 280 bytes captured (2240 bits) on interface lo, id 0						
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)						
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1						
Transmission Control Protocol, Src Port: 5003, Dst Port: 38420, Seq: 172, Ack: 206, Len: 214						
[2 Reassembled TCP Segments (385 bytes): #176(171), #178(214)]						
Hypertext Transfer Protocol						
JavaScript Object Notation: application/json						

Comportamento teste_cliente.py

O objetivo do teste_cliente.py é testar o funcionamento das APIs REST que foram criadas (todas acima), a criação foi um sucesso. Também foi feito muitos testes com Curl para realizar inserções diretas sem precisar colocá-las no código (como vemos em todas as APIs acima).

PRINT 1 e 2: Os dois prints de terminal abaixo demonstram o sucesso da criação do teste_cliente.py, que foi criado com o objetivo de testar se as APIs REST foram um sucesso, ou seja, há uma variação de métodos a serem testados, como GET, POST, PUT e DELETE (como observado em todo o roteiro acima).

```
(venv) allan-samuel@allan-samuel-VirtualBox:~/python$ python3 teste_cliente.py
{"id": 102,
 "nome": "Mouse Sem Fio",
 "preco": 120.5
}

--- REQUISIÇÃO POST: http://localhost:5002/produtos ---
Enviando Dados: {'nome': 'Monitor Ultra Wide', 'preco': 1850.99}
Status Code: 201
Recurso Criado (201 Created):
{
  "id": 103,
  "nome": "Monitor Ultra Wide",
  "preco": 1850.99
}

--- REQUISIÇÃO POST: http://localhost:5002/produtos ---
Enviando Dados: {'preco': 50.0}
Status Code: 400
Erro de Validação (400 Bad Request): Dados inválidos. 'nome' e 'preco' são obrigatórios.

--- REQUISIÇÃO GET: http://localhost:5002/produtos ---
Status Code: 200
Dados Recebidos:
[
  {
    "id": 101,
    "nome": "Notebook",
    "preco": 3500.0
  },
  {
    "id": 102,
    "nome": "Mouse Sem Fio",
    "preco": 120.5
  },
  {
    "id": 103,
    "nome": "Monitor Ultra Wide",
    "preco": 1850.99
  }
]
```

```
(venv) allan-samuel@allan-samuel-VirtualBox:~/python$ python3 teste_cliente.py

--- REQUISIÇÃO GET: http://localhost:5001/usuarios ---
Status Code: 200
Dados Recebidos:
[
  {
    "email": "alice.s@exemplo.com",
    "id": 1,
    "nome": "Alice Silva"
  },
  {
    "email": "bruno.c@exemplo.com",
    "id": 2,
    "nome": "Bruno Costa"
  }
]

--- REQUISIÇÃO POST: http://localhost:5001/usuarios ---
Enviando Dados: {'nome': 'Carlos Vianna', 'email': 'carlos.v@exemplo.com'}
Status Code: 201
Recurso Criado (201 Created):
{
  "email": "carlos.v@exemplo.com",
  "id": 3,
  "nome": "Carlos Vianna"
}

--- REQUISIÇÃO POST: http://localhost:5001/usuarios ---
Enviando Dados: {'nome': 'Daniel'}
Status Code: 400
Erro de Validação (400 Bad Request): Dados inválidos. 'nome' e 'email' são obrigatórios.

--- REQUISIÇÃO GET: http://localhost:5001/usuarios ---
Status Code: 200
Dados Recebidos:
[
  {
    "email": "alice.s@exemplo.com",
    "id": 1,
    "nome": "Alice Silva"
  },
  {
    "email": "bruno.c@exemplo.com",
    "id": 2,
    "nome": "Bruno Costa"
  },
  {
    "email": "carlos.v@exemplo.com",
    "id": 3,
    "nome": "Carlos Vianna"
  }
]
```

Conclusão

Sem dúvidas o laboratório mais difícil até o momento, a quantidade de trabalho posta nele foi de uma grande utilidade, entender como funciona uma API REST na prática é, de fato, uma experiência marcante, mas verificar o sucesso das

aplicações é mais marcante ainda (positivamente falando). Dessa maneira, esse laboratório é e será de uma grande utilidade no avanço do conhecimento em redes.